Nr. acces ... $= 10^8 \times 0,1 + (\%\text{ Reads} \times \text{Read Miss Penalty} + \%\text{ Writes} \times$
$\times \text{Write Miss Penalty}) + 10^8 \times 0,9 \times \underbrace{0,3}_{\%\text{Writes}} \times \underbrace{\text{Write Hit Penalty}}_{=1}$

Read Miss Penalty = Write Miss Penalty =

$= \underbrace{2 \text{ BUS Reads}}_{\text{Allocate}} + \underbrace{0,35 \cdot 2 \text{ BUS Writes}}_{\text{Update for Main Memory}}$

CURS 13

28.05.2019

b) Nr. acces $= \underbrace{10^8 \text{ words/second}}_{\text{Frequency of memory references}} \times \underbrace{0,1}_{\text{Miss Rate}} \times (0,3 \text{ Write Miss Penalty} +$
$+ 0,7 \text{ Read Miss Penalty}) + 10^8 \times 0,9 \times 0,3 \text{ Write Hit Penalty}$

Read Miss Penalty = 2 BUS Reads (nu mai updateaza)

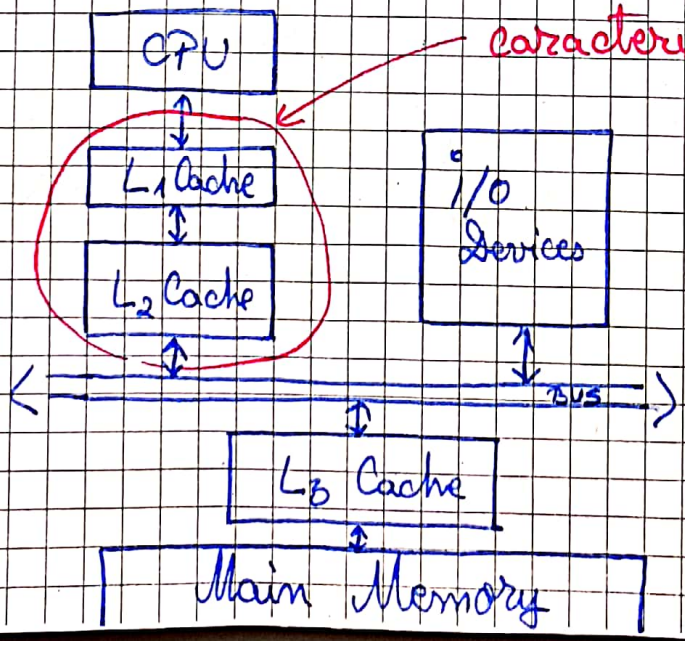
Write Miss Penalty = 2 BUS Reads + 1 BUS Write =
= 3 BUS Accesses

Write Hit Penalty = 1 BUS Write = 1 BUS Access

Atentie: Nr. acces: $10^8 \cdot 0,1 \,(0,3 \cdot 3 + 0,7 \cdot 2) + 10^8 \cdot 0,9 \cdot 0,3 \cdot 1 =$
$= 10^7 \cdot 2,3 + 2,7 \cdot 10^7 = 5 \cdot 10^7$

$\%\text{ Bus Used}_{WT} = \frac{10^7 \cdot 5}{10^9} = \frac{5}{100} = 5\,\%$

3.9. Reducing miss rate with multi-level caches



Scanned by CamScanner

Example 1: Se dă un sist. de memorie cu 2 nivele de cache unde CPI ideal = 1,0 clock cycle.

Clock Rate = 4 GHz

MM access time = 100 ns (incluzând Miss Penalty în MM)

Miss Rate per instruction = 2% (în L_1)

Se câte ori va fi mai rapid procesorul dacă adăugăm L_2 cache cu un timp de acces de 5 ns și pt. hit-uri și pt. miss-uri și e suficient de mare cât să reducă Miss Rate la MM la 0,5% din cazuri.

♥ Doar cu L1:

CPU time original = IC × (CPI ideal + Memory accesses per instruction × Miss Rate × Miss Penalty) × CCT

Misses per instruction = 0,2

CPU time original = IC × (1 + 0,02 × 400) × 0,25 ns = 2,25 ns × IC

$CCT = \frac{1}{4 \cdot 10^9 Hz} = 0,25 \text{ ns} \left(= \frac{1}{\text{Clock Rate}}\right)$

$\text{Miss Penalty} = \left\lceil \frac{100 \text{ ns}}{0,25 \text{ ns}} \right\rceil = 400$ c.c. (clock cycles)

↳ găsește info în MM.

♥ L_1 cu L_2:

$\text{CPU}_{\text{time 2L}} = IC \times [CPI_{\text{ideal}} + (0,02 - 0,005) \times 20 + 0,005 \times (400 \text{ c.c} + 20 \text{ c.c})] \times 0,25 \text{ ns} = 0,85 \text{ ns} \times IC$

(Miss Penalty L_2 = 20; (0,02 − 0,005): info nu nimerește în L_1 și hit-uie în L_2; 0,005: info nu nimerește nici în L_1, nici în L_2)

$\text{Miss Penalty}_{L_2} = \left\lceil \frac{5 \text{ ns}}{0,25 \text{ ns}} \right\rceil = 20$ c.c

↳ miss penalty când nu găsește în L_1 și găsește în L_2

Scanned by CamScanner

Așadar: $\frac{\text{CPU time orig}}{\text{CPU time 2L}} = \frac{2,25}{0,85} = 2,64$

3.10. Virtual Memory Mechanism

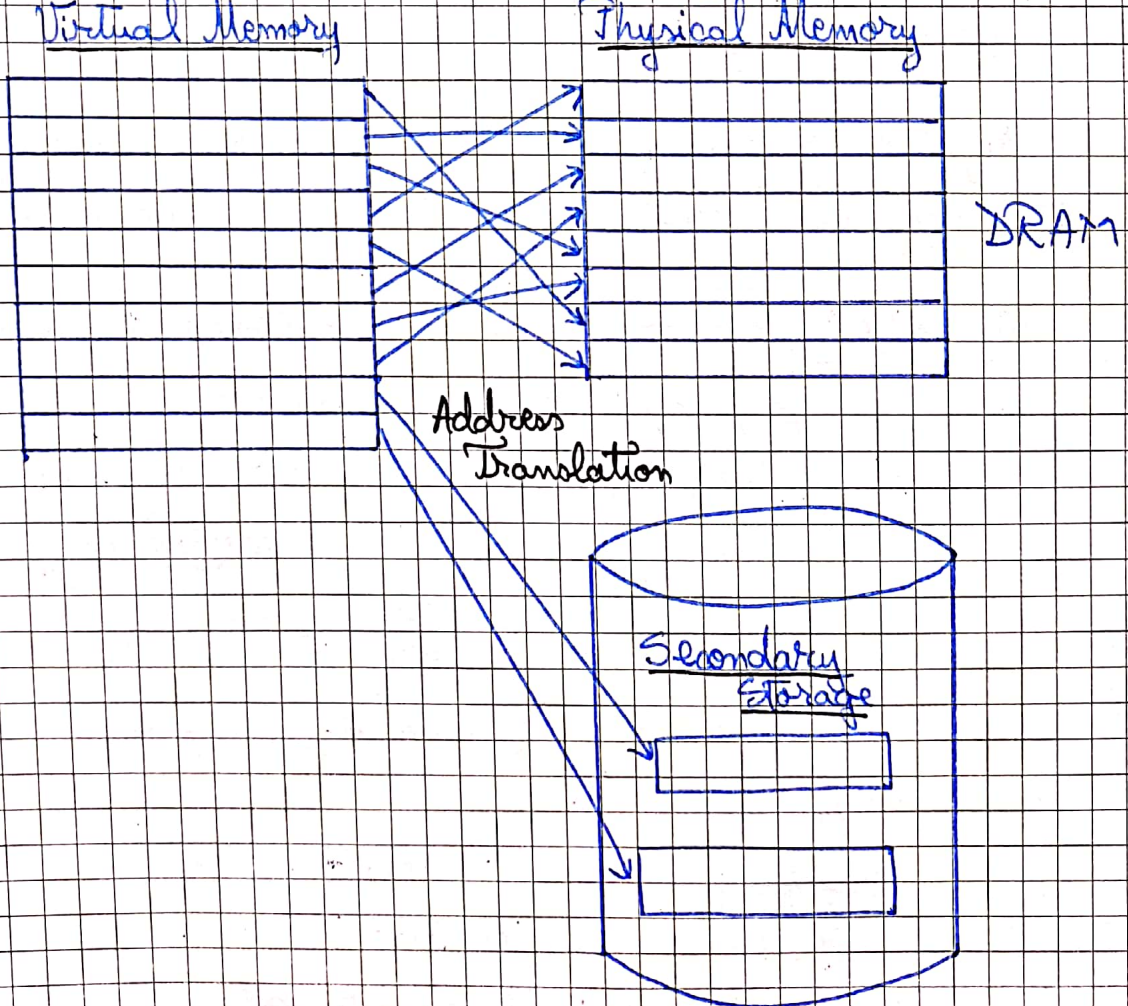
Virtual Machines, probleme puse pt. implementarea lor:

1. Protect processes from one another
2. Share the limited memory among processes (VMs)

Pentru asta, programatorii foloseau overlay, dar acum se foloseste o masină virtuală ce foloseste un spatiu de memorie virtuală (SMV) ce urmează a fi mapat pe adresele fizice.

SMV se fragmentează în: → Segments (variable size)
→ Pages (fixed size) ← cel care ne interesează pe noi

> Mecanismul de memorie virtuală paginat:

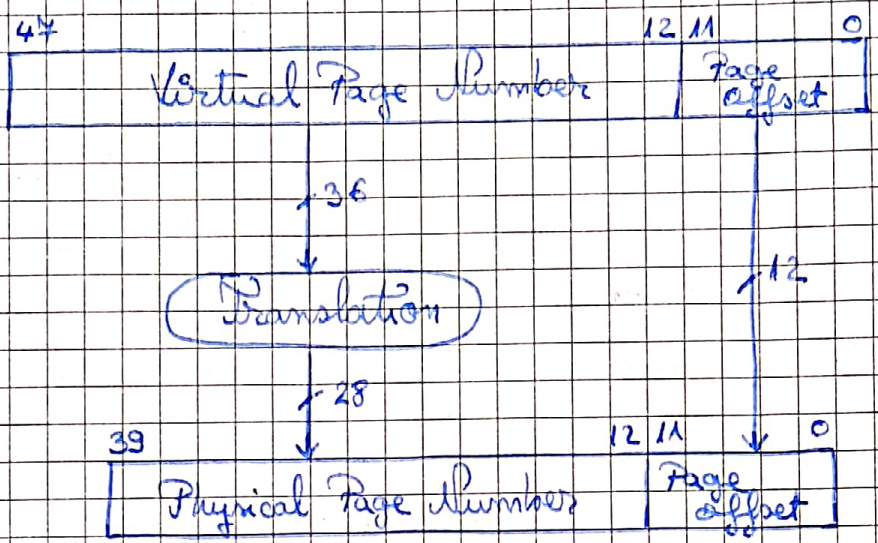


Scanned by CamScanner

- Exemplu pt. Address Translation pe ARM V8:

64 Bit Address space
16 MSB are unused

64 - 16 = 48 - 12 = 36
40 - 12 = 28



Page size = 2^{12} B = 4 KiB (B = Bytes) ← pe 12 biți exprimăm adresa unui octet
Physical memory size = 2^{40} B = 1 TiB = 2^{28} pages = 256 Mi Pages
Virtual memory size = 2^{48} B = 256 TiB = 2^{36} pages = 64 Gi Pages

Paginile virtuale se mapează în memoria fizică doar când procesul este activ (posedă CPU la un moment dat)

Denumiri: page → granule
page fault → MMU exception (Memory Management Unit)
↳ când pagina nu e în MM și trebuie adusă de pe disc prin mecanismul de memory swapping

Problemă luată în considerare pt. construirea unui MMU: Misses cost a lot! (Secondary storage is 100 000 x slower than MM). Rezolvări:

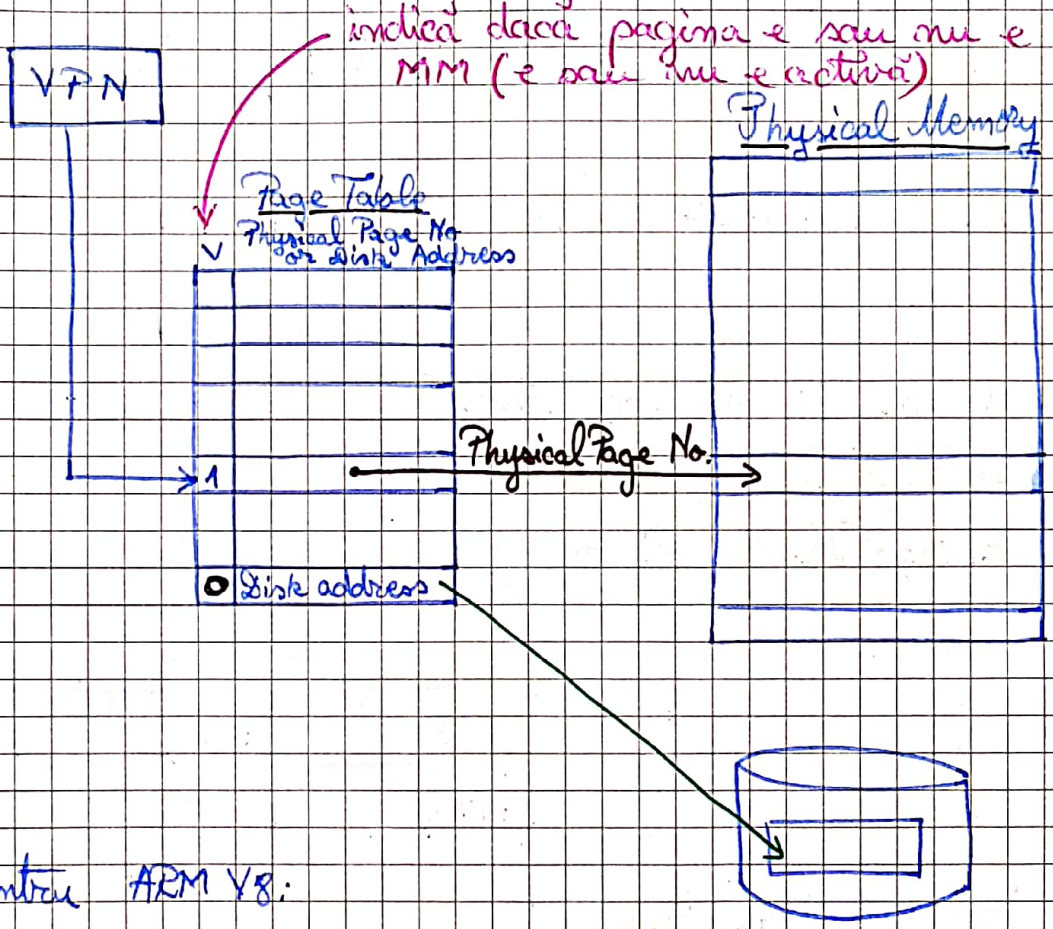
(1) relatively big pages: 4 KiB ÷ 64 KiB
Servers → 32 KiB ÷ 64 KiB
Embedded → 1 KiB ÷ 4 KiB

Scanned by CamScanner

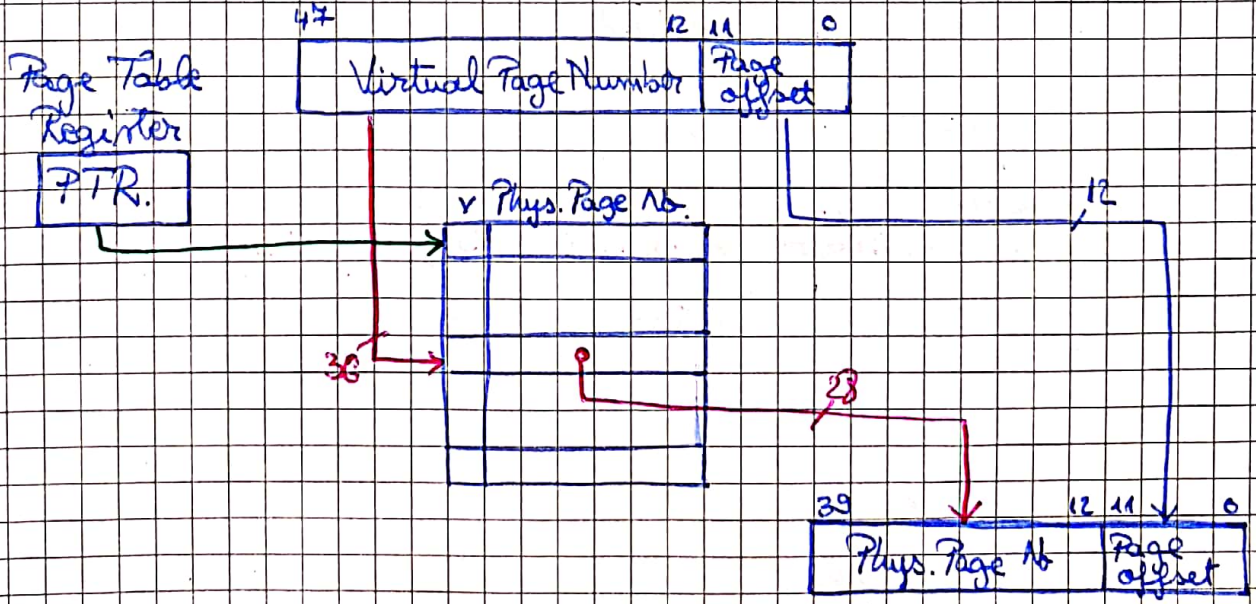
② WT e imposibil -> Folosim doar WB

③ Replacement is decided in software (nu mai folosesc metode random) - approximate LRU

④ Nu îți poți permite să cauți tag-uri ->
=> Folosim doar full-associativity mapping cu Page Tables



♥ Pentru ARM V8:



Scanned by CamScanner

Problems: (1) Page Table Size

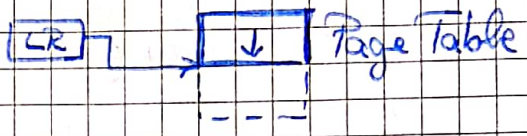
ARM - 1 PT Entry = 64 bits = 2^3 B

$2^{36} \times 2^3 = 2^{39} = 0{,}5$ TiB

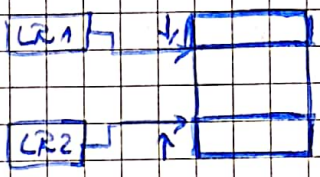
(2) Page Table Reads (Page Table is placed in the Physical Memory)

Solutions:

(1) A. Limit register



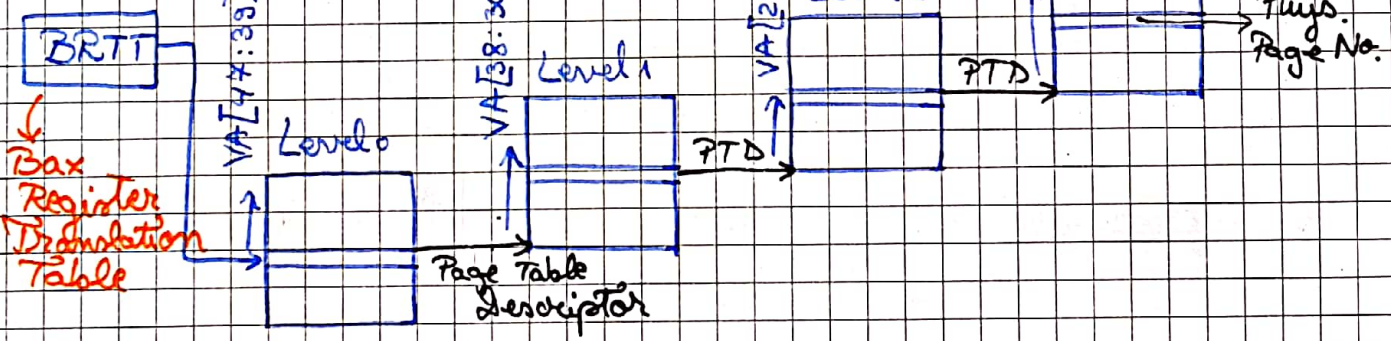
B. Heap & Stack growth



C. Hash (inverse page table)

D. Page the Page Table

E. Page Table Hierarchy

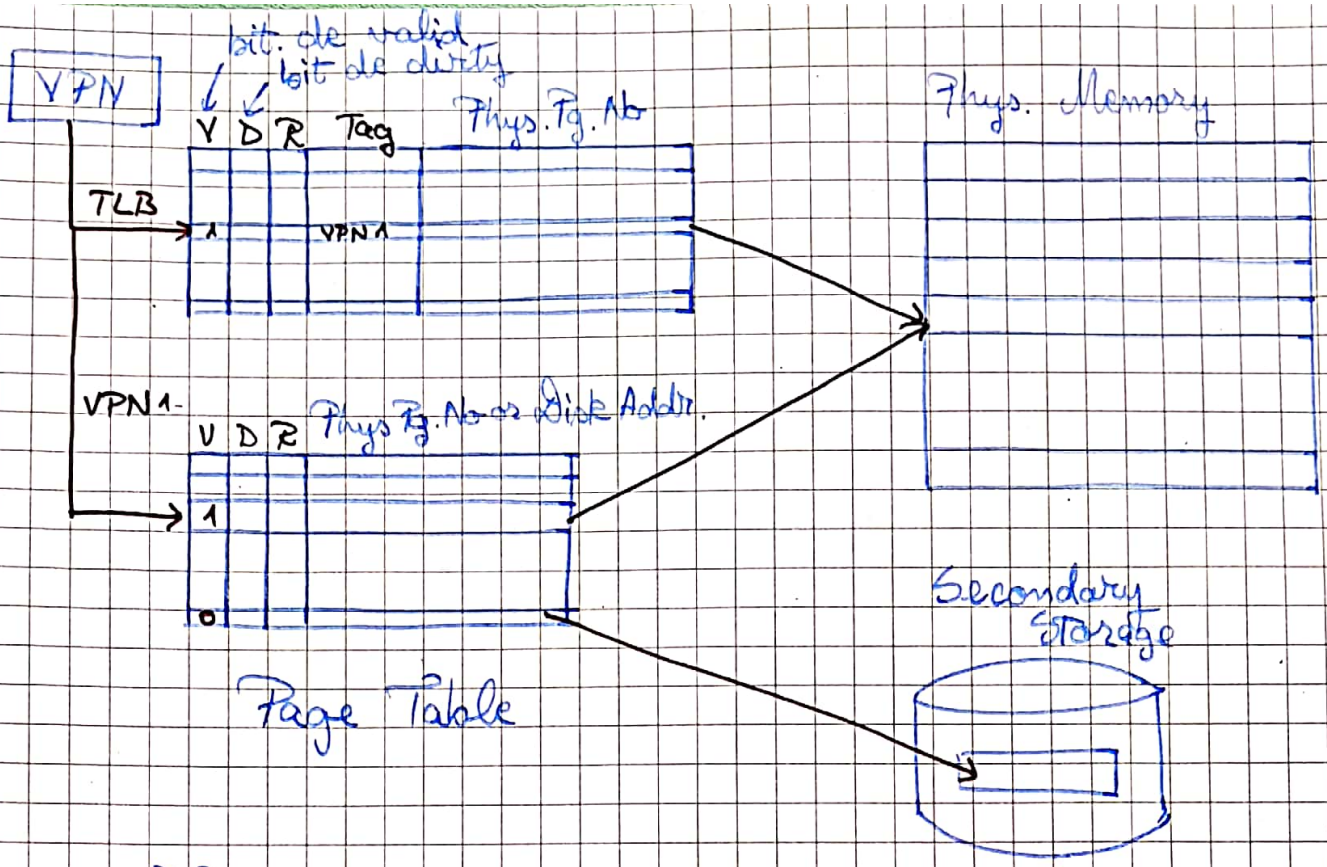


$2^9 \times 2^3 B = 2^{12} B = 4$ KiB

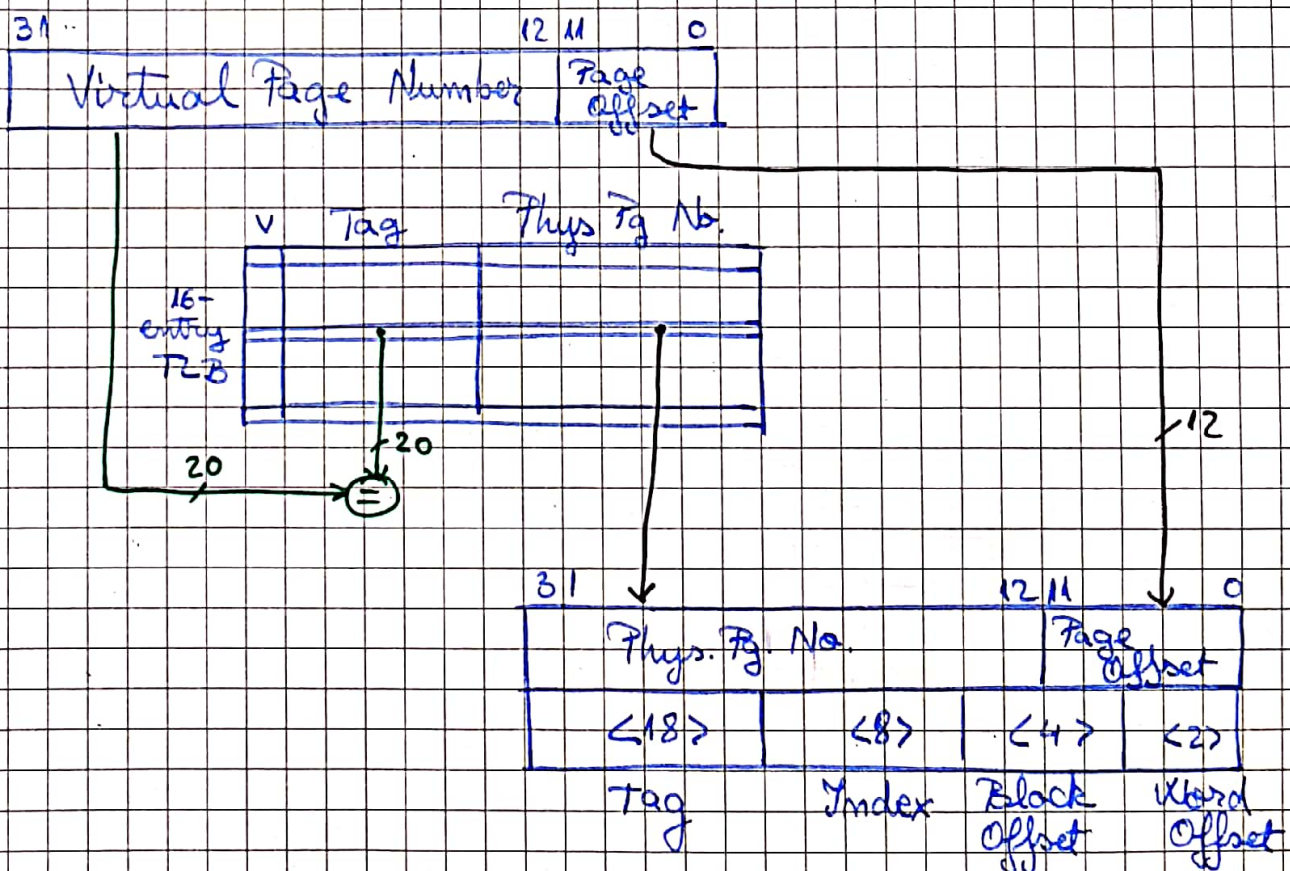
(2) Translation Lookaside Buffer (TLB)

- Cache the Page Table

Scanned by CamScanner



Fast MATH



Scanned by CamScanner